

Project Proposal

AgentDesign: Hermes Profile Architecture

Math & Librarian Profiles — Design Documenta

Mehdi Ghasemi

Project Proposal

September 2026

Executive Summary

This document records the complete design of the two Hermes agent profiles that power my mathematical research workflow — `math` and `librarian` — and specifies an abstraction layer under which either profile can be re-pointed at any future math project without modification. All facts are extracted from the live configuration on `/home/mehdi/.hermes/profiles/` (config version 49, September 2026); every claim in Sections 3–8 is verifiable against the files listed in Section 10.

The architecture resolves into five layers: a shared *Hermes core*, an isolated *profile system*, a *skill catalog* (57–59 skills, capability-gated per profile), an *MCP bridge* (13 registered servers, one generic CLI-adaptor pattern covering 8 of them), and an *external service plane* on the local network. The math profile is a full-stack researcher with a 5.9 GB sandboxed toolchain environment; the librarian profile is a capability-degraded derivative that shares every registration but disables all computational skills and adds two ingestion skills of its own.

Two structural findings drive the abstraction design: (i) the librarian profile’s MCP servers import scripts from the math profile’s directory tree, so it cannot exist without the math profile; and (ii) both profiles contain a stale reference to `IreneRewrite`, deleted when that worktree was merged into `Irene` on 2026-09-18. Section 9 proposes a capability/manifest model — persona templates, path interpolation, a shared adapter, a service registry, and per-project binding files — that removes every identified coupling in five concrete steps.

Contents

1 Objectives and Scope	3
2 Layered Architecture	3
3 Profile System: Structure Comparison	4
4 Skill System: Design for General Math Projects	6
5 MCP Bridge	8
6 Tool Stack and Service Plane	9
7 The Research Workflow	11
8 Identified Couplings and Defects	13
9 Toward an Abstraction Layer	13
10 Roadmap for AgentDesign Itself	16

I Objectives and Scope

The `AgentDesign` project has three objectives:

1. **Document the as-is architecture.** Record, with file-level evidence, how the math and librarian profiles are composed: directory layout, configuration sections, skill catalogs, MCP server registrations, external services, scheduled jobs, and state stores. This is Sections 2–7.
2. **Identify the coupling points.** Enumerate every place where a profile depends on the specific current project layout — absolute paths, cross-profile imports, hardcoded endpoints, and stale references — so that the design of record distinguishes what is load-bearing from what is incidental (Section 8).
3. **Specify an abstraction layer.** Define how profiles become reusable personas: independent of any particular math project, instantiable for a new project with zero or one line of configuration, and testable against a capability contract (Section 9).

The scope is the two active research profiles. The remaining four registered profiles (`default`, `coder`, `advisor`, `trader`) are out of scope; their existence is noted only in Section 3. This project does not modify any live profile configuration — it is a documentation and design artifact.

2 Layered Architecture

Figure 1 gives the visual layout of the infrastructure under which both profiles operate. Five layers are distinguished; each lower layer is shared, each upper layer is isolated per profile at `~/.hermes/profiles/<name>/`.

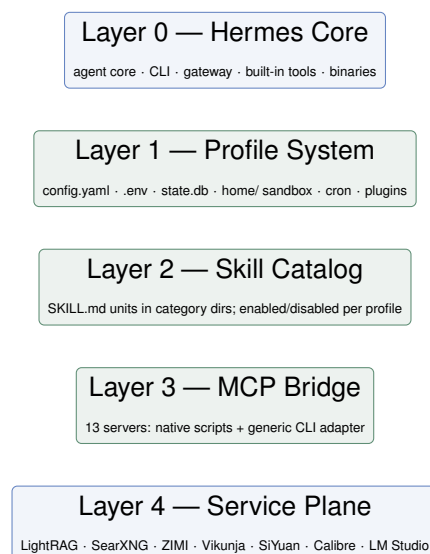


Figure 1: Five-layer profile architecture. Layers 2–3 are the capability surface of a profile; layers 0 and 4 are shared infrastructure. Arrows indicate the direction in which each layer is configured by, or depends on, the layer above it.

Layer 0 — Hermes Core

The core lives at `~/.hermes/hermes-agent/`: agent runtime, CLI (profile switching, tool registry), platform gateway (Telegram/WhatsApp/webhook adapters), built-in tools (terminal, browser, vision, file I/O, web

search, kanban, memory) and the MCP discovery layer (`tools/mcp_tool_*.py`). Shared binaries (tirth security scanner, `uv`, Python 3.14, Node 26, ripgrep, Chromium) live in `~/.hermes/tools/`; all MCP stdio servers run from a single shared virtualenv at `~/.hermes/mcp-servers/.venv/`.

Layer 1 — Profile System

Each profile is an isolated directory with its own configuration, credentials, state and sandbox. The two profiles under study:

- **math** (8.4 GB; 493 sessions): full-stack mathematical researcher. Working directory `/home/mehdi/Code/Python/`. Carries a 5.9 GB sandboxed home environment with the complete local toolchain (Section 3). Active Telegram gateway; five cron jobs registered (two enabled). Kanban orchestrator and default assignee is `math`.
- **librarian** (174 MB; 8 sessions): literature manager — “manage and maintain books and articles.” Working directory `/home/mehdi/Code/`. Empty `home/` stub, no cron jobs, no gateway platforms. Shares the same kanban assignment (`math`), which is itself a coupling noted in Section 8. It is also the only profile with an active metrics sender: its `telemetry.shared_metrics` block has both `enabled` and `send` set to `true`; `math`’s config carries no telemetry block at all.

Both profiles run the same local model: `qwen3.8-27b` via LM Studio at `http://localhost:1234/v1`, with an `on_session_start` hook (`~/.hermes/agent-hooks/ensure-lmstudio.sh`) that guarantees the model server is up before each session. Long-term memory in both profiles is backed by Honcho (`memory.provider: honcho`, service at port 8008).

Layer 2 — Skill Catalog

Skills are Markdown-defined capability units (`SKILL.md` with YAML frontmatter), organized in category directories, optionally carrying scripts and credential files. Each profile holds 57 skill units across nine named categories plus a handful of top-level skills (union: 59); the full per-profile inventory is in `references/skill_inventory.md`; Section 4 summarizes the design-relevant parts.

Layer 3 — MCP Bridge

Skills reach services either directly (terminal/shell) or through Model Context Protocol servers registered under `mcp_servers`: in each profile’s config; every registered tool is exposed to the model as `mcp__<server>__<tool>` (e.g. `mcp__vikunja__tasks__create`). Section 5 documents the two server families and the cross-profile dependency this layer creates.

Layer 4 — Service Plane

A fixed set of self-hosted services on the local network (host `192.168.1.70`, DDNS fallback `mghasemi.ddns.net`), plus two SaaS APIs and one local inference server. Table 5 lists them with their role in the workflow.

3 Profile System: Structure Comparison

Table 1 compares the structural elements of the two profiles. All values were read directly from `~/.hermes/profiles/<p>/` in September 2026.

Table 1: Structural comparison, math vs. librarian profile.

Element	math	librarian
config version	49	49
Model / provider	qwen3.8-27b / lmstudio	qwen3.8-27b / lmstudio (+explicit chat_completions api mode)
Terminal cwd	/home/mehdi/Code/Python/	/home/mehdi/Code/
.env size	52 lines, 49 keys	45 lines, 42 keys (strict subset of math; no Telegram/WhatsApp/OpenRouter)
Skills (SKILL.md)	57 across 9 named categories + top-level (9 further empty category shells)	same shape; 2 skills unique to each (librarian: calibre-library-ingestion, djvu-to-pdf; math: exact-symbolic-gate-scripts, irene-rewrite-dev)
skills.disabled entries	12	28 (superset: all computational + dev skills)
MCP servers registered	13	13 (identical names; two differ in content, Table 4)
Sandbox home/	5.9 GB toolchain env (Table 2)	empty directory
bin/	tirth, uv, uvx	tirth only
scripts/	mcp_cli_adapter.py + wiki pipeline (8 further .py)	absent
plugins/ dir	hermes-android, vikunja, wiki-browser	empty
Enabled plugins (config)	vikunja, wiki-browser	vikunja, wiki-browser (installed under shared ~/.hermes/plugins/)
Cron jobs	5 registered (2 enabled: arXiv scan every 10 080 min; Wiki Maintenance weekly Sundays 03:00 — named “bi-weekly” but the cron is 0 3 * * 0)	none (no jobs.json; ticker state only)
Sessions in state.db	493	8
Kanban orchestrator / assignee	math / math	math / math
Code execution mode	project	project
Approvals mode	off (600 s timeout)	off
Compression threshold	—	0.6
Telemetry block	absent	present (shared_metrics, send=true — the only profile transmitting metrics)
Desktop repo scan roots	/home/mehdi/Code/Python/	/home/mehdi/Code/
Total profile size	8.4 GB	174 MB

The librarian is a capability-degraded derivative of math, not an independent design: it inherits every skill directory and MCP registration, then suppresses the computational surface via the `skills.disabled` list. It also carries two skills that exist only in its own tree — `calibre-library-ingestion` and `djvu-to-pdf` (productivity category) — which is where its genuine specialization lives.

The math sandbox home environment, the single largest structural asymmetry:

Table 2: Contents of `~/.hermes/profiles/math/home/` (5.9 GB). The profile home is a partial toolchain plus package caches: Lean 4 lives here; SageMath and TeXLive resolve to host-level installs (`/usr/bin/pdflatex`, `miniconda3/envs/sage`).

Directory	Role
<code>.elan/</code> (2.9 GB)	Lean 4 toolchain <code>leanprover-lean4--v4.31.0</code> used by the lean4 MCP server
<code>.rustup/</code> (1.4 GB), <code>.cargo/</code> <code>.cache/</code> (1.2 GB)	Rust stable-x86_64 toolchain plus registry and download caches Package/browser caches: ms-playwright (674 MB), pip (517 MB) — not a toolchain
<code>.npm/</code> (355 MB)	npm cache for the sandboxed node 26 environment
<code>.notebooklm/</code> (21 MB)	Dedicated Chrome profile for NotebookLM grounded synthesis
<code>.texlive2023/</code> (14 MB)	texmf-var font/format caches only; actual TeXLive is host-level
<code>.sage/</code> , <code>.maxima/</code>	SageMath runtime state directories (binary resolves to host conda env)
<code>wiki/</code> (28 KB)	Small local wiki with SCHEMA.md index — distinct from the main 149 MB wiki at <code>/home/mehdi/Code/wiki</code>
<code>.ssh/</code> , <code>.gitconfig</code> , <code>.bashrc</code>	Identity and shell configuration of the sandboxed user
<code>pb.py</code> , <code>wiki_ingest.py</code> , <code>pb_token.txt</code>	PocketBase + wiki ingestion pipeline scripts (9 files)

Two facts matter for the abstraction design. First, the 5.9 GB figure is inflated by roughly 1.7 GB of disposable package caches (`.cache/`, `.npm/`); the load-bearing toolchain state is Lean under `.elan/` and Rust under `.rustup/`. Second, SageMath and TeXLive were never actually installed in this home — the sagemath MCP server falls through its candidate list to a host conda env, and `pdflatex` comes from `/usr/bin` — so the profile home is not a self-contained computation environment despite looking like one.

4 Skill System: Design for General Math Projects

Skills are the unit of reusability. Each `SKILL.md` carries a trigger description in its frontmatter, an imperative body, optional scripts, and — where needed — a co-located credential file (`.env`; two exist under `math` — `skills/productivity/siyuan/.env` and `plugins/vikunja/dashboard/.env` — and one under `librarian`). The catalog is shared between the two profiles (same 24 top-level skill directories: nine sub-skill categories, six top-level skills in `math` / five in `librarian`, and nine empty category shells); only activation differs. Table 3 summarizes by category; **E/D/–** = enabled / disabled / absent per profile (counts from `references/skill_inventory.md`).

Table 3: Skill catalog by category (E/D counts of skills present in the profile tree).

Category	Counts (math / lib)	Design role
research	24E/1D 18E/6D	Core literature + computation surface: <code>math-research-workflow</code> (the 6-stage pipeline), <code>sagemath-mcp</code> , <code>sympy-mcp</code> , <code>lean4</code> , <code>lean4-goedel-agent</code> , <code>lightrag</code> , <code>searxng</code> , <code>zimi</code> , <code>wolfram-alpha</code> , <code>academic-research-hub</code> , <code>arxiv</code> , <code>wiki pipeline skills</code> ; <code>math-only: exact-symbolic-gate-scripts</code>
productivity	7E/1D 7E/3D (8 / 10 total)	Knowledge management: <code>vikunja</code> , <code>siyuan</code> , <code>zotero</code> , <code>calibre</code> (disabled in both), <code>latex-manuscript</code> , <code>project-bib</code> , <code>ocr-and-documents</code> ; <code>librarian adds calibre-library-ingestion</code> , <code>djvu-to-pdf</code>
mlops	3E/1D 2E/2D	Local inference ops: <code>lmstudio-configuration</code> , <code>pocketbase</code> , <code>dsdp-extension-workflow</code> (librarian disabled)
software-development	6E 4E/2D	Hermes self-extensibility, planning, subagent orchestration, desktop DOM inspection
github / devops / autonomous-ai-agents / web	e.g. github 1E/1D 0E/2D	Repo ops, SDLC review (lib disabled), <code>merge-reconciler</code> , <code>blocked-page recovery</code>
social-media	0E/1D 1E/0D	<code>reddit-reading</code> — inverted gate: off for math, on for librarian
top-level skills	6E 3E/2D	<code>differential-sdp</code> , <code>ghasemi-latex-style</code> , <code>hermes-plugin-management</code> , <code>semantic-scholar</code> , <code>workspace-doc-hierarchy</code> ; <code>math adds irene-rewrite-dev</code> (librarian disables it)

Three observations are design-relevant:

1. **The math research surface is exactly the set of skills a generic math project needs:** decompose → literature → compute → verify → write, implemented by `math-research-workflow`, which names 18 sibling skills in its frontmatter — 17 of them live catalog entries, the eighteenth (`simplerag-memory`) dead. This set, not the full catalog, defines the reusable “mathematician” capability contract (Section 9).

2. **The librarian's disabled list is a negative specification of that same contract:** every computational skill (sagemath-mcp, sympy-mcp, lean4, lean4-goedel-agent, scientific-coding), the DSDP skills, and the Irene/dev skills are turned off. Capability gating by negation works today but is brittle in a subtler way than it looks: entries resolve by *leaf name* against the profile's skill tree, which includes the curator tombstone directory `skills/.archive/`. All 40 disabled names (12 math + 28 librarian) therefore resolve to a real `SKILL.md`; none dangles. But 6 of math's and 9 of the librarian's entries resolve *only* under `.archive/` — they are residue from skills the curator has already retired (e.g. `docx`, `opencode`, `test-driven-development`; for the librarian also `irene-rewrite-dev`, which exists as an *active* top-level skill in math while pointing at a librarian tombstone). Because resolution is name-based, any of these 15 stale entries would silently disable a same-named skill if it were ever reinstalled. One inversion deserves note: `reddit-reading` is disabled for math but enabled for librarian — the only skill whose gate runs opposite to every other difference between the two profiles.
3. **One reference is stale:** `math-research-workflow` names `simplerag-memory` ten times (frontmatter plus nine body references), eight of them shell invocations of a nonexistent `simplerag_client.sh`; no such skill exists in either profile tree or in the bundled pool.

5 MCP Bridge

Thirteen MCP servers are registered per profile, all launched with the shared interpreter `~/.hermes/mcp-servers/.venv/bin/python`. They fall into three families:

Table 4: MCP server registrations. “Pattern” distinguishes native servers from generic CLI-adapter wraps; the script path is as written in each profile's config (identical strings for both profiles except where noted).

Server	Pattern	Script (path relative to)	Tools
sagemath	native	<code>profiles/math/skills/research/sagemath-mcp/</code>	10
lean4	native	<code>profiles/math/skills/research/lean4/</code>	4
searxng	native	<code>profiles/math/skills/research/searxng/scripts/</code>	1
vikunja	CLI adapter	<code>tool: profiles/math/skills/productivity/vikunja/</code>	15
wolfram-alpha	CLI adapter	<code>.../research/wolfram-alpha/</code>	3
lightrag-query	CLI adapter	<code>.../research/lightrag/</code>	3
siyuan	CLI adapter	<code>.../productivity/siyuan/</code>	7
zimi	CLI adapter	<code>.../research/zimi/</code>	4
zotero	CLI adapter	<code>.../productivity/zotero/</code>	7
scientific-coding	CLI adapter	<code>.../research/scientific-coding/</code> (CPU/mem sandbox limits via env)	4
latex-manuscript	CLI adapter	<code>.../productivity/latex-manuscript/</code> (compile, AST check, glossary, notation audit, diff-patch)	5
hermes-irene	direct script	<code>Code/Python/IreneRewrite/scripts/hermes_mcp_tool.py</code> — path no longer exists (worktree merged into Irene, folder deleted 2026-09-18; file never committed to git)	—
calibre	native (librarian only, active)	math: env block only, no command; librarian: <code>Code/Python/tools/calibre-mcp/server.py</code> with its own venv, base URL :9876, timeout 300 s	12

The **generic CLI-adapter pattern** is the load-bearing design of this layer. One script — `profiles/math/scripts/mcp_cli_adapter.py` — wraps any argparse-based Python CLI tool as a stdio MCP server: it introspects the target's parser at

startup, exposes each subcommand as an MCP tool with JSON-schema arguments, and reconstructs command lines from tool calls. Eight of thirteen servers use it; adding a new service-backed skill therefore requires only a thin CLI script plus a config entry (URL/token via env), no protocol code.

Two cross-profile facts follow directly from the table:

- Every path in every MCP registration — including all of the librarian's — points into the **math profile's directory tree**: 8 `TOOL_SCRIPT` values, 3 native server scripts, and the adapter itself (`profiles/math/scripts/mcp_cli_adapter.py`). The librarian has no `scripts/` directory of its own. Deleting or moving the math profile would silently break every tool-bridged capability of the librarian.
- Both profiles register `hermes-irene` against a script inside `IreneRewrite`, which was merged into `Irene` and deleted on 2026-09-18; in math it is `enabled: false` (dormant), in librarian `true` (broken at startup). The correct target after the merge would live under `/home/mehdi/Code/Python/Irene/`, where no such script has yet been created.

6 Tool Stack and Service Plane

Table 5 lists the services both profiles reach, grouped by role in the research workflow. All self-hosted endpoints share host `192.168.1.70` (fallback `mghasemi.ddns.net`); credentials live in each profile's `.env` (42 shared keys; 7 math-only, all messaging/auxiliary: Telegram bot token and allowlist, WhatsApp mode fields, OpenRouter key).

Table 5: Service plane. Both profiles share the same registrations; differences noted.

Service	Endpoint	Role in work-flow	MCP
LightRAG	:9621 (API key)	Graph-RAG over the curated literature lake: ingest + multi-hop query	yes × 1 profile
SearXNG	:5050	Federated web metasearch, structured JSON output	yes
ZIMI	:8899	Offline encyclopedia (Wikipedia ZIM) for archival grounding before live search	yes
Vikunja	:3456 (token)	Task ledger: hypothesis trees, phase tracking, cross-session	yes + plugin

The **four integration channels** a skill may use, in the preference order codified by `math-research-workflow`:

1. *MCP native / adapter tools* — fastest path into services; no shell involved.
2. *Terminal invocations of CLI scripts* bundled with skills (e.g. the arXiv research script, wiki ingestion pipeline).
3. *Built-in Hermes tools* — web search/extraction, file I/O, browser automation.
4. *Local binaries from the sandbox home* (SageMath, Lean 4 via `elan/rustup`, TeXLive) for computation and compilation that must stay on-machine.

7 The Research Workflow

The designated workflow for math projects is the `math-research-workflow` skill (v1.6), a single-agent adaptation of the MathAgent multi-agent system with Graph-of-Agents message passing and Reflexion quality gates. Its six stages, each gated by a 5-check Reflexion gate:

Stage	Activity	Primary tools
1. Problem decomposition	Restate the problem; open Vikunja tasks and a hypothesis tree (conjecture = parent, attack branches = children with status)	vikunja MCP
2. Literature synthesis	Local-first retrieval cascade: LightRAG → academic-research-hub → Semantic Scholar → SearXNG → ZIMI; every citation passes the <i>Citation Verification Gate</i> (arXiv API / Crossref / S2 before any bibliographic fact is recorded)	lightrag, searxng, zimi, s2 skill
3. Computation	Deterministic-first: SymPy → SageMath → Wolfram Alpha → sandboxed scientific-coding; Error-Feedback Loop with a 3-retry budget, exact prior error carried into each retry	sympy-mcp, sagemath-mcp, wolfram-alpha, scientific-coding
4. Formal verification	Three-stage Lean pipeline: formalize → compile clean (<code>lean4_check</code> against an explicit Mathlib project) → statement-fidelity diff vs. the informal claim; only then is a theorem ledger-marked “proven & Lean-verified”	lean4, lean4-goedel-agent
5. Manuscript production	Drafter → Enhancer → Reviewer pass over LaTeX with the ghasemi-latex-style anti-pattern checklist and notation audit	latex-manuscript MCP, ghasemi-latex-style
6. Reflection & close-out	Claims ledger updated; failures promoted to skill pitfalls or project reports; project README + AGENTS.md status synced in the same commit	—

Supporting state: a per-project failure scratchpad at `~/.hermes/profiles/math/scratchpad/<problem>/failures.md`, per-project verified-source registries for citation dedup, and the cron jobs (weekly arXiv scan in the five-topic overlap set; bi-weekly wiki maintenance) that feed Stage 2 continuously. The Irene package (`/home/mehdi/Code/Python/Irene/`) is the shared numerical backend for polynomial optimization experiments across projects (moment/SOS hierarchies, SONC relaxations), invoked from Irene's venv rather than through MCP — the intended `hermes-irene` bridge being the broken one documented in Section 5.

8 Identified Couplings and Defects

Ranked by severity (number of stars), with file-level evidence:

Table 6: Coupling points binding profiles to the current project layout.

Coupling	Evidence	Severity
Cross-profile script import (librarian → math)	All 8 adapter <code>TOOL_SCRIPT</code> values, 3 native server scripts and <code>mcp_cli_adapter.py</code> itself resolve under <code>profiles/math/</code> ; librarian has no scripts/ dir	*****
Stale IreneRewrite path in both profiles	<code>hermes-irene</code> args point at <code>Code/Python/IreneRewrite/scripts/hermes_mcp_tool.py</code> ; directory deleted 2026-09-18; enabled for librarian	*****
Absolute skill paths in MCP config	Every registration hardcodes the full home path; no interpolation exists anywhere in <code>config.yaml</code>	****
Hardcoded service endpoints	42 shared <code>.env</code> keys embed <code>192.168.1.70:port</code> ; no name-based resolution	***
Project-pinned working dirs	<code>terminal.cwd</code> , <code>desktop.repo_scan_roots</code> and <code>code_execution.mode: project</code> reference specific project trees; kanban orchestrator fixed to <code>math</code> in both profiles	***
Sandbox toolchain baked into math home	5.9 GB of Sage/TeX/Lean/Rust state inside the profile dir — a project-agnostic capability shipped as profile baggage	**
Stale disabled entries (curator tombstones)	15 of the 40 <code>skills.disabled</code> names (6 math + 9 librarian, e.g. <code>context-budget-audit</code> , <code>local-model-delegation</code>) resolve only under <code>.archive/</code> ; they silently disable a same-named skill if ever reinstalled	**
Orphaned skill reference	<code>simplerag-memory</code> named by <code>math-research-workflow</code> ; absent from both catalogs	*
Asymmetric calibre registration	math: <code>env-only</code> block; librarian: full server — same name, different meaning per profile	*

9 Toward an Abstraction Layer

The goal is a design in which *profiles are personas and projects are bindings*. A profile declares **what it can do** (capabilities) and *how it behaves* (persona); a project declares **where things live** (paths, endpoints, ledgers). Nothing in today's configuration should need editing to attach the existing math or librarian profile to a new math project.

A. Capability-gated skill activation (replaces negative lists)

Each `SKILL.md` gains one frontmatter field:

```
1 requires: [computation, formal-proof] # empty list = always available
```

and each profile declares a positive capability set instead of a disabled list:

Persona	Capabilities
mathematician (today: math)	literature, computation, formal-proof, manuscript, knowledge-mgmt, optimization-backends
librarian	literature, ingestion, knowledge-mgmt, manuscripts-light

The loader activates a skill iff `requires` $\subseteq$ `persona.capabilities`. The librarian's current 28-entry negative list disappears; tombstone names (entries that resolve only under `.archive/`) become impossible to carry forward, since a capability gate cannot name a skill the persona does not declare; and a new persona is one YAML block, not a diff of every registration.

B. Path interpolation + shared adapter (removes ★★★★★ coupling)

Introduce `${PROFILE_HOME}` and `${HERMES_ROOT}` tokens in `mcp_servers` entries, mirroring the `{baseDir}` mechanism skills already use. Concretely:

1. Move the generic adapter to a shared location: `~/.hermes/tools/mcp_cli_adapter.py` (next to `tirith/uv`, which are already cross-profile). Adapter-wrapped tool scripts move with their skills — they do, since `TOOL_SCRIPT` paths become `${PROFILE_HOME}/skills/...`
2. Each profile's config then references only its own tree; the librarian stops importing from math. Verification is trivial: no path in a profile's config may contain another profile's name.

C. Service registry (removes ★★★ endpoint coupling)

A `services.yaml` naming services abstractly, with endpoints resolved from `env` at load time:

```
1 # ~/.hermes/services.yaml
2 lightrag: { url: ${LIGHTRAG_URL}, key: ${LIGHTRAG_API_KEY} }
3 searxng: { url: ${SEARXNG_URL} }
4 zimi: { url: ${ZIMI_URL} }
5 vikunja: { url: ${VIKUNJA_URL}, token: ${VIKUNJA_TOKEN} }
6 siyuan: { url: ${SIYUAN_URL}, token: ${SIYUAN_TOKEN} }
7 calibre: { url: ${CALIBRE_BASE_URL} }
8 honcho: { url: ${HONCHO_BASE_URL} }
```

MCP entries then reference `$services.lightrag.url` etc. The `.env` files remain the secret store (never committed), but IPs move out of configuration logic into environment, so a service migration is an `env` edit, not a per-profile config surgery. This also fixes the calibre asymmetry: one definition, activated by capability `ingestion`.

D. Toolchain bundle (removes ** baggage coupling)

The 5.9GB sandbox home is a *capability artifact* (computation + formal-proof + manuscript), not profile state. Split it: keep identity files (`.gitconfig`, `.ssh`, `.bashrc`) in the profile; extract the toolchain into an installable bundle `math-toolchain.tar` or a Docker/ conda-spec manifest that any persona declaring the relevant capabilities can provision on demand — and that a project without formal-proof needs (e.g. pure numerical work) never pays for.

E. Project binding files (the actual project-independence mechanism)

A new math project is then attached by a single file at its root:

```

1 # /home/mehdi/Code/<NewProject>/agent.yaml
2 profiles: [math, librarian]
3 working_dir: .
4 vikunja_project: <id>          # task ledger target
5 lightRAG_collection: <project> # literature namespace
6 sage_env: sage                # toolchain bundle component (conda env name)

```

Profiles discover projects by scanning their `repo_scan_roots`; a profile picks up a project's binding at session start and overrides `terminal.cwd`, ledger targets, and collection names from it. Nothing inside the profiles is edited when the project changes — which is exactly the property today's coupling list (Table 6) shows to be absent.

F. Remediation prerequisites

Before any of A–E is built, four live defects should be fixed in place (each a one-line or one-file change): repoint `hermes-irene` at the merged Irene tree (or disable it for the librarian), prune the 15 tombstone-only names from the disabled lists, remove `simplerag-memory` from `math-research-workflow`'s related-skill list, and give math a real calibre server command or delete its env-only block. These are recorded here as open work items; none is applied by this documentation project.

Instantiation model, end state

Figure 2 summarizes the target relationship between templates, personas and projects.

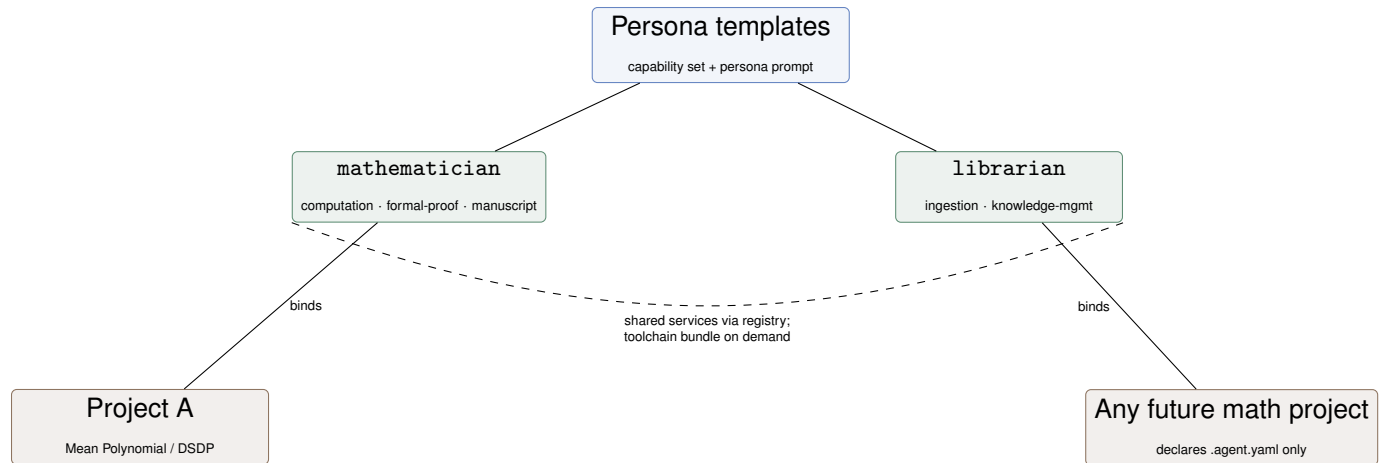


Figure 2: Target model: personas are instantiated from templates with a capability set; projects bind to personas through root-level binding files. No profile-internal configuration changes when the project changes.

10 Roadmap for AgentDesign Itself

Phase	Activity	Status
I. Exploration	Read every config, skill tree, DB and script; extract data programmatically (references/extract_profile_data.py)	done (2026-09-29)
II. Documentation	This document + skill_inventory.md + extracted JSON in data/	this deliverable
III. Remediation PRs	The four one-line fixes of Section 9(F), as separate commits against the live profiles	open
IV. Abstraction pilot	Implement capability frontmatter + loader gate for one skill pair (e.g. sagemath-mcp under a test persona); verify librarian behavior is unchanged	open
V. Migration	Shared adapter move, services.yaml, toolchain bundle; then retire disabled lists	open

Sources and Verification

All facts in this document were read from the following locations on 2026-09-29 (config version 49); no claim was taken from memory:

- `~/.hermes/profiles/{math,librarian}/`: config.yaml, .env (values redacted in data/profile_comparison.json), profile.yaml, state.db, cron/jobs.json, scripts/, plugins/, home/, full skill trees.
- `~/.hermes/plugins/` (shared plugin installs), `~/.hermes/mcp-servers/.venv/`.
- Extraction script and output: AgentDesign/references/extract_profile_data.py, AgentDesign/data/profile_comparison.json, skill inventory: AgentDesign/references/skill_inventory.md.
- Irene merge date (2026-09-18) and absence of hermes_mcp_tool.py under the merged tree: filesystem check.

Redaction note: API keys, tokens and passwords appear in the source `.env` files but are masked as `***` everywhere in this project's data artifacts; URLs, ports and paths are recorded verbatim because they carry no secret value.